Project Sanri: System Architecture

Next-Generation Multi-Layered Cyber-Deception Framework

ABSTRACT

Sanri is an advanced, multi-layered cyber-deception platform engineered to detect, confuse, and analyze malicious actors in real time. Unlike traditional static honeypots relying on hardcoded responses, Sanri provides a high-interaction, dynamic deception surface. By blending low-latency Rust proxying, AI-driven generative web interfaces, dynamic isolation environments, and automated Z3-based formal analysis, it intercepts adversarial behavior and synthesizes high-fidelity threat intelligence.

1. Introduction & Project Overview

The modern cyber threat landscape demands adaptive defense mechanisms. Static emulation fails against sophisticated adversaries who easily fingerprint traditional honeypots. Project Sanri addresses this by establishing a living, generative deception surface. It operates not merely as a logging mechanism, but as an active engagement platform that dynamically adjusts to incoming payloads, safely isolating attackers while extracting actionable telemetry for automated rule synthesis.

2. Microservices & Polyglot Architecture

The architecture employs a decoupled, polyglot approach designed for ultra-low latency and scalable reasoning. The core routing and deep packet inspection (DPI) are implemented in Rust for memory safety and zero-cost abstractions. Heavy computational lifting—such as the Z3 SMT Analysis Engine and the LLM Generative Engine—is built as asynchronous Python / FastAPI microservices. This separation of concerns via lightweight TCP/IPC ensures that complex threat reasoning does not bottleneck the primary gateway.

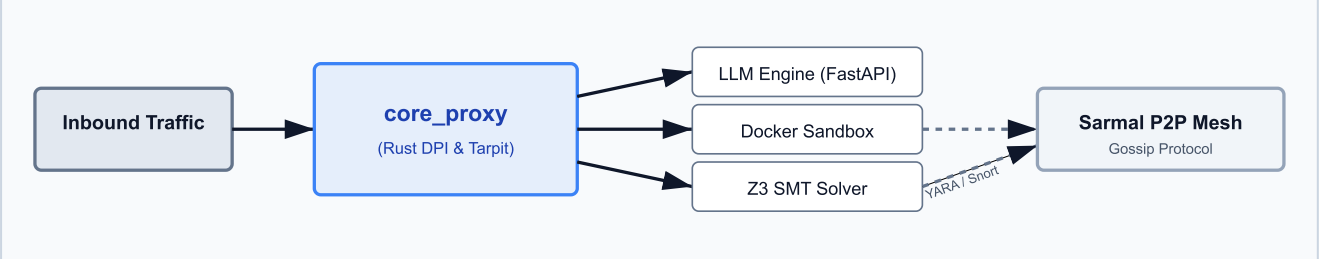


Figure 1: Decoupled Polyglot Architecture of Project Sanri

2.1 Core Proxy & Deep Packet Inspection (DPI)

The primary ingress point of the system is `core_proxy`, a high-performance gateway written in Rust. Operating at the protocol level, it safely inspects and categorizes incoming connection attempts before routing them to the appropriate deception environment.

LAYER 7 DPI & HEURISTIC ROUTING

Moving beyond simple port-forwarding, the `core_proxy` acts as a Layer 7 (Application Layer) DPI engine. Upon intercepting the initial raw payload bytes, it utilizes a custom *detect_protocol* heuristic module. By analyzing structural byte sequences (e.g., SSH banner exchanges, TLS handshakes, or MySQL protocol headers), it automatically categorizes the threat before routing. This ensures the payload is forwarded to the exact corresponding deception environment without breaking the active protocol handshake.

TARPIT & ARTIFICIAL LATENCY INJECTION

To combat automated vulnerability scanning and mass-exploitation frameworks, the proxy engine implements strategic "tarpitting". It deliberately injects asynchronous artificial latency (e.g., 1.5s for SSH negotiation, 2.5s for Database handshakes) during connection establishment. This severely degraded response time exponentially slows down automated attack scanners (such as Nmap or Masscan) and buys critical compute time for the dynamic target containers to spin up transparently.

2.2 Sarmal P2P Distributed Defense Layer

Sarmal is Sanri's distributed defense layer. Built on a peer-to-peer (P2P) mesh network model, Sarmal allows independent edge nodes of Sanri to exchange threat telemetry, newly observed malicious IPs, and synthesized detection rules globally without requiring a centralized, vulnerable controlling node.

ADVANCED GOSSIP PROTOCOL & SID TRACKING

To ensure rapid threat propagation without causing network congestion or infinite routing loops, Sarmal utilizes an advanced Gossip Protocol. When a node receives a newly synthesized threat signature, it checks a localized ledger using a unique Signature ID (SID). If the SID is known, the packet is silently dropped. If it is novel, the node logs the threat and asynchronously broadcasts (gossips) the payload to its peers, guaranteeing network-wide immunity within milliseconds of a single 0-day detection.

3. Key Architectural Features

The integration of advanced deception tactics with rigorous formal analysis yields a platform capable of handling complex, multi-stage attacks.

Feature Component	Technology Stack	Function & Impact
Dynamic Orchestration	Docker API, Rust	Spawns fresh, isolated target containers on-demand upon connection detection, minimizing baseline system resource utilization.
Deep Packet Inspection (DPI)	Rust	Analyzes structural byte sequences to automatically categorize Layer 7 threats without breaking active protocol handshakes.
Generative Interfaces	Python, FastAPI, LLM	Uses real-time prompt-driven response generation to emulate complex, proprietary web applications without static templates.
Rule Synthesis	Python, FastAPI, Z3	Analyzes recorded exploit payloads and symbolic execution paths to synthesize structured YARA and Snort rules automatically.
Distributed Defense	Sarmal P2P Ledger	Propagates newly generated indicators of compromise (IOCs) across all nodes in the mesh to coordinate instant blocklists.
Latency Injection	Rust Async Delays	Injects artificial latency during connection establishment, slowing down automated scanners and allowing containers to boot.

3.1 Z3 SMT Solver & Automated Threat Analysis

When an attacker delivers an exploit payload or unknown executable into a sandbox container, Sanri passes the observed execution traces and binary properties to its analysis pipeline powered by the Z3 Theorem Prover (SMT Solver). By evaluating path conditions and symbolic variables, the Z3-driven engine reconstructing execution requirements for zero-day vulnerabilities, extracts invariant byte patterns, and automatically outputs production-ready YARA and Snort signature files.

4. Deployment & Infrastructure

Sanri is designed for rapid containerized deployment in diverse network environments, ranging from cloud VPCs to edge networks. Every honeypot service target runs inside restricted Docker containers using non-root namespaces, capped resource limits (cgroups), and read-only root filesystems where applicable to prevent sandbox escapes.

The host system exposes only the `core_proxy` listening ports, keeping internal routing interfaces, the LLM pipeline, and the Sarmal P2P control plane completely segregated from public access. This guarantees that the deception layer cannot be used as a pivot point into internal infrastructure.